Expt. No. 1a | Date / / | Page No. 1

Name of Experiment..... Experiment Result .....

LAB CYCLE 1

Aim:

a) To study and verify the truth table of logic gates
b) Realise basic gates using universal gates
c) Simplify the given expression and realize it using gates.

a) To study and verify the truth table of logic gates

Components required:

i) 7404 - NOT
ii) 7432 - OR
iii) 7408 - AND
iv) 7400 - NAND
v) 7402 - NOR
vi) 7486 - XOR
vii) 74266 - XNOR

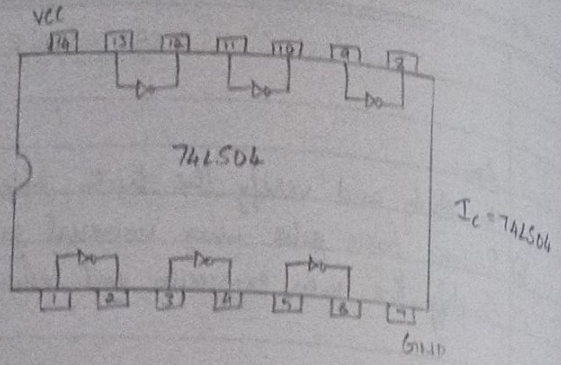
Theory

A digital circuit with one or more input signals but only one output signal is called as logic gates. Since a logic gate is a switching circuit, its output can have only one of the 2 possible states, either a higher voltage (1) or a low voltage (0).

Truth table is the one which shows the conditions at the input and output of a logic gate in binary form.

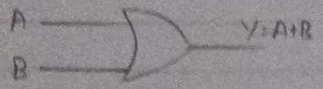
(i) NOT Gate:

A	Y
0	1
1	0

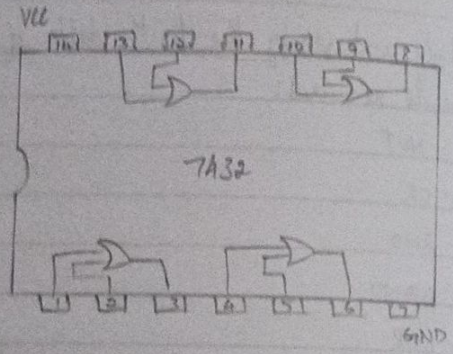


I_c = 74LS04

(ii) OR Gate:



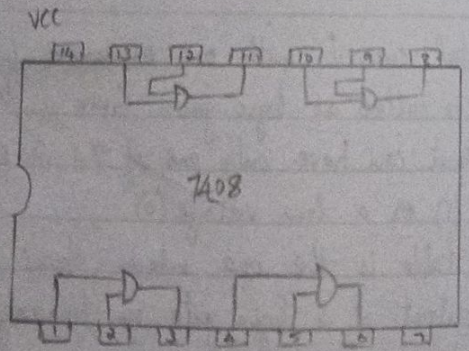
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



I_c = 7432

(iii) AND Gate

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



I_c = 7408

Expt. No. 1a Date / / Page No. 2

Name of Experiment ..... Experiment Result .....

Procedure

i) NOT gate: 7404

In digital logic, an inverter or NOT gate is a logic gate which implements logical negation. It outputs a bit opposite of the bit that is put into it.

The hex inverter is an integrated circuit that contains six inverters. The 7404 IC has 14 pins out of which 12 are used by input and outputs of the six inverters.

ii) OR gate: 7432

The OR gate is a digital logic circuit that implements logical disjunction. The OR gate outputs 'TRUE' if any one of its inputs are true, otherwise it outputs 'FALSE'.

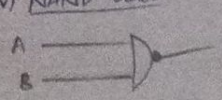
The 74LS32 IC has 14 pins out of which 12 are used for input and output signals. It has 4 OR gates, hence it is also called as quad IC.

iii) AND gate: 7408

The AND gate is a basic digital logic gate that implements logical conjunction (∧) from mathematical logic. A high output (1) results only if all the inputs to the AND gate are high (1).

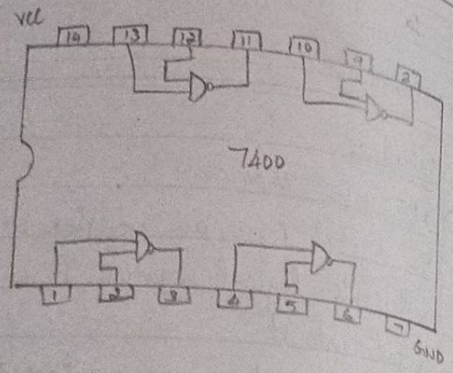
IC 7408 has also 14 pin configuration and consists of 4 logical gates. 12 out of 14 pins are used for input and output signals.

(iv) NAND Gate

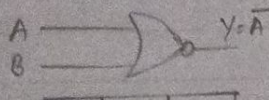


A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

I_c = 7400

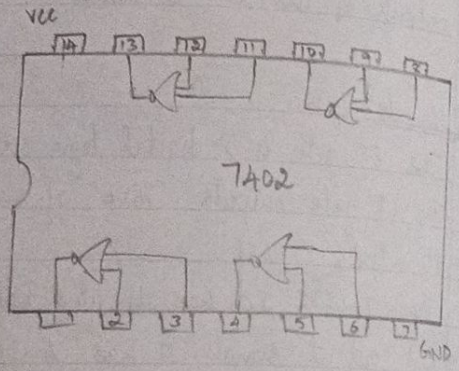


(v) NOR Gate

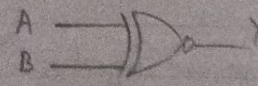


A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

I_c = 7402

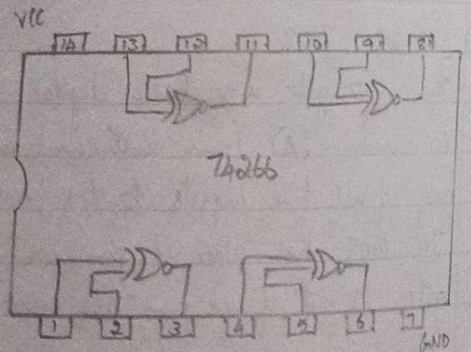


(vi) XNOR Gate



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

I_c = 74266



Expt. No. 1a Date / / Page No. 3

Name of Experiment..... Experiment Result .....

i) NAND gate: 7400

In digital electronics, a NAND gate (NOT AND) is a logic gate which produces an output which is false only if all its inputs are true. Thus its output is compliment to that of an AND gate.

The NAND gate is significant because only boolean function can be implemented by using a combination of NAND gates.

The standard 2-input NAND gate is 7400 which is a quad-two input NAND gate, i.e. it contains four logic gates in it.

ii) NOR gate: 7402

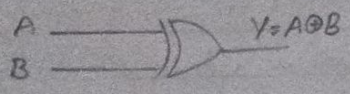
The NOR gate is a digital logic gate that implements logical NOR gate. A high output (1) results if both the inputs to the gate are low (0). If one or both input are high (1), a low output (0) results. NOR is the result of negation of the OR operation 7402 is a quad 2-input NOR gate.

vi) XNOR gate: 74266

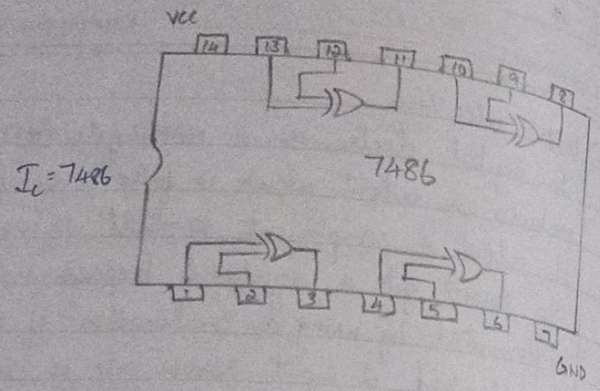
The XNOR gate is a digital logic gate whose function is the logical complement of the exclusive OR (XOR) gate. It is equivalent to the logical connective ($\leftrightarrow$) from mathematical logic, also known as the material biconditional.

XNOR gates are represented in most TTL and CMOS IC. The standard TTL IC is the 74266. It includes four independent, two input XNOR gate.

(vii) XOR Gate:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



Expt. No. 1a Date / / Page No. 4

Name of Experiment..... Experiment Result .....

vii) XOR gate: 7486

XOR gate is a digital logic gate that gives a true output when the number of true input is odd. An XOR gate implements an exclusive (↔) from mathematical logic. A true output results if one and one of the inputs to the gate is true.

The most common standard chip is TTL 7486 quad dual input XOR gate.

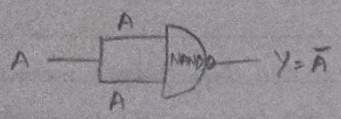
Procedure

1) Collect the required IC's & mount them on Ic trainee kit with the grove facing to the top.
2) Connect the inputs, outputs, VCC, ground using patch cords
3) Perform the experiment and verify truth table.

Result

All the results / truth tables for the logic gates have been verified through this experiment.

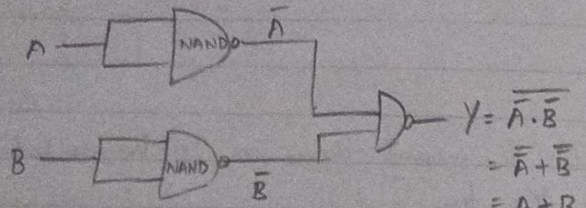
(i) NOT using NAND:



$[\bar{A}.A = \bar{A} + \bar{A} = \bar{A}]$

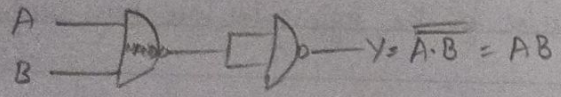
A	Y
0	1
1	0

(ii) OR using NAND:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

(iii) AND using NAND:



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Page No. 5

Expt. No. 16 Date / /

Name of Experiment..... Experiment Result .....

b) Realize basic gates using universal gates

Components required:

1) IC 7400 : NAND
2) IC 7402 : NOR

(i) NOT gate using NAND:

To realize the NOT gate using the NAND gate, we have to join the 2 input terminals of the NAND gate to form a single input terminal of the NOT gate and the output of the NOT gate is taken from the output terminal of the NAND gate.

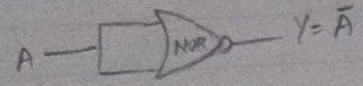
(ii) OR gate using NAND:

To obtain OR gate from NAND gate, we feed input A into a NAND with the input joined together to obtain a NOT(A). We feed input B into another NAND gate NAND with the input joined together to obtain NOT(B). Now NOT(A) and NOT(B) are fed into separate input of another NAND gate. Hence it becomes equal to OR gate

(iii) AND gate using NAND:

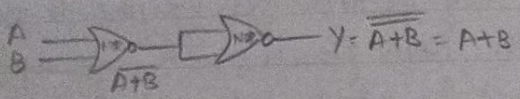
We feed inputs A and B into the NAND gates, when the output says Y is fed into another NAND gate with the 2 inputs joined together to obtain an output, say Z which is NOT.

(i) NOT using NOR:



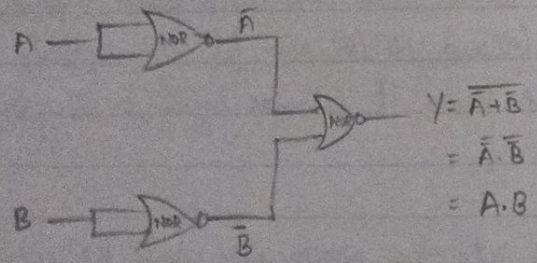
A	Y
0	1
1	0

(ii) OR using NOR:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

(iii) AND using NOR:



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Expt. No. 16 Date / / Page No. 6

Name of Experiment ..... Experiment Result .....

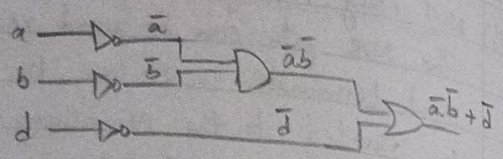
(i) NOT using NOR gate:
The NOR gate can be used as NOT gate by connecting all its input terminals except one to logic 0 and applying the signal to be inverted to the remaining terminal. This configuration of NOR gates as NOT gate is refered to as a controlled inverter.

(ii) OR using NOR gate:
NOR gate is actually a combination of 2 logic gates: OR gate followed by NOT gate. So its output is complement of the output of an OR gate. This gate can have maximum two inputs, output is always one.

(iii) AND using NOR gate:
For the realization of an AND gate from NOR gate, we require 3 NOR gates. The first two gates are used to complement the input variable A and B, and the third NOR gate is used to procedure an output equivalent to the AND operation.

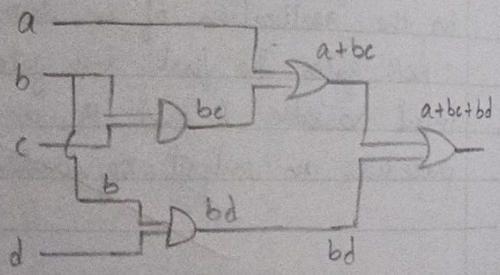
1.

	00 $\bar{c}\bar{d}$	01 $\bar{c}d$	11 cd	10 $c\bar{d}$
00 $\bar{a}\bar{b}$	0 1	1	3 1	2 1
01 $\bar{a}b$	4 X	5 0	7 0	6 1
11 ab	12 X	13 0	15 0	14 1
10 $a\bar{b}$	8 X	9 0	11 0	10 1



2.

	00 $\bar{c}\bar{d}$	01 $\bar{c}d$	11 cd	10 $c\bar{d}$
00 $\bar{a}\bar{b}$	0 0	1	3 0	2 0
01 $\bar{a}b$	4 0	5 1	7 1	6 1
11 ab	12 X	13 X	15 X	14 X
10 $a\bar{b}$	8 1	9 1	11 X	10 X



Expt. No. 1c Date / / Page No. 7

Name of Experiment..... Experiment Result .....

c) Simplify the given expression and realize it using gates

Components required

1) Patch cords
2) IC 7400
3) IC 7402
4) IC 7404
5) IC 7432
6) IC 7408
7) IC 7410
8) IC 7411

Theory:

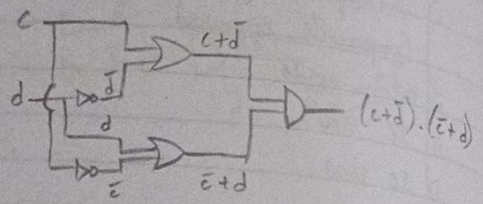
Any boolean expression can be simplified using boolean laws to reduce number of gates in circuit and realize it using gates.

Procedure:

1) Simplify given expression or modify them till the stage we cant modify it further.
2) Write truth table of expression and correct IC.
3) Draw circuit diagram for the expression.
4) Build the circuit and check the results using Truth Table.

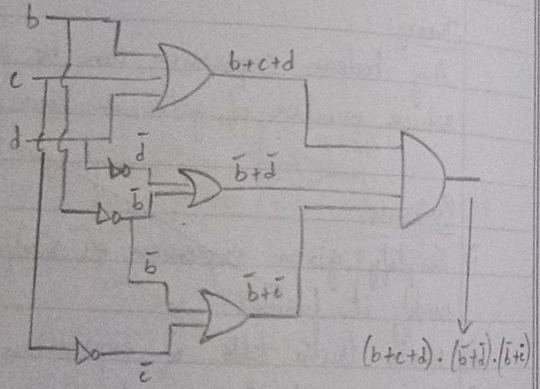
3.

	00 c+d	01 c+d̄	11 c̄+d̄	10 c̄+d
00 a+b	0: 1	1: 0	3: 1	2: 0
01 a+b̄	4: 1	5: 0	7: 1	6: 0
11 ā+b̄	12: X	13: X	15: X	14: X
10 ā+b	8: 1	9: 0	11: X	10: X



4.

	00 c+d	01 c+d̄	11 c̄+d̄	10 c̄+d
00 a+b	0: 0	1: 1	3: 1	2: 1
01 a+b̄	4: 1	5: 0	7: 0	6: 0
11 ā+b̄	12: X	13: X	15: X	14: X
10 ā+b	8: 0	9: 1	11: X	10: X



Expt. No. 1c Date / / Page No. 8

Name of Experiment..... Experiment Result .....

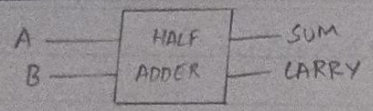
1. $f(a,b,c,d) = \Sigma(0,1,2,3,6,10,14) + dc(4,8,12)$
   $f = \bar{a}\bar{b} + \bar{d}$

2. $f(a,b,c,d) = \Sigma(5,6,7,8,9) + dc(10,11,12,13,14,15)$
   $f = a + bc + b\bar{d}$

3. $f(a,b,c,d) = \pi(1,2,5,6,9) + dc(10,11,12,13,14,15)$
   $f = (c+\bar{d}) \cdot (\bar{c}+d)$

4. $f(a,b,c,d) = \pi(0,5,6,7,8) + dc(10,11,12,13,14,15)$
   $f = (b+c+d) \cdot (\bar{b}+\bar{d}) \cdot (\bar{b}+\bar{c})$

Half adder



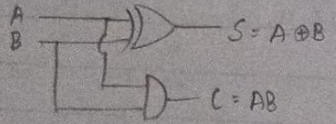
Truth table:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

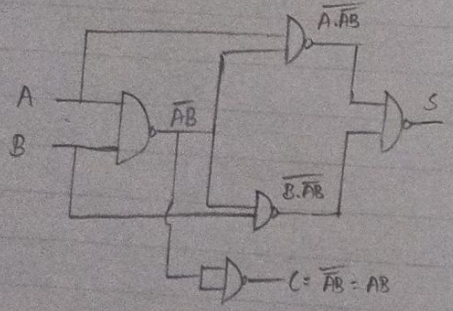
$Sum = \bar{A}B + A\bar{B}$

$= A \oplus B$

$Carry = AB$



Implementation of Half adder using NAND gate



$S = \overline{(\overline{A.\bar{A}B}).(\overline{B.\bar{A}B})}$

$= \overline{A.\bar{A}B} + \overline{\overline{B.\bar{A}B}}$

$= A.\bar{A}B + B.\bar{A}B$

$= A(\bar{A}+\bar{B}) + B(\bar{A}+\bar{B})$

$= A\bar{A} + A\bar{B} + B\bar{A} + B\bar{B}$

$= A\bar{B} + \bar{A}B$

$= A \oplus B$

Page No. 9

Expt. No. 2a Date / /

Name of Experiment ..... Experiment Result .....

LAB CYCLE - 2

Design and realize

(a) Half adder:

A half adder is a combinational circuit that performs the addition of two single-bit binary numbers. It has two inputs, typically labeled AAA and BBB, and two outputs: the sum (SSS) and the carry (CCC).

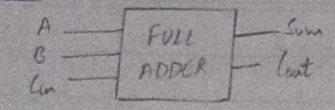
- Sum (S): The sum output is the result of the XOR operation between the two input bits.

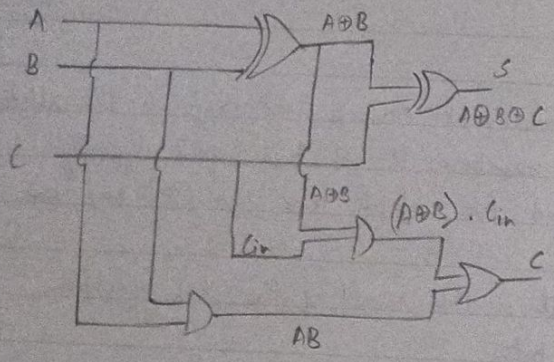
  $S = A \oplus B$

- Carry (C): The carry output is the result of the AND operation between the two input bits.

  $C = A \cdot B$

Full adder

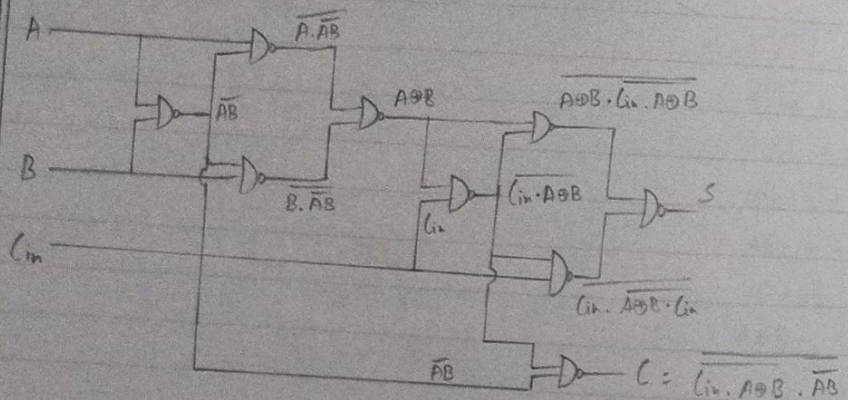




Truth table:

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Implementation of Full adder using only NAND gates



Expt. No. 6 Date / / Page No. 10

Name of Experiment..... Experiment Result .....

(b) Full Adder

A full adder is a combinational circuit that performs the addition of three single-bit binary numbers: two significant bits (AAA and BBB) and a carry-in. It has three inputs and two outputs: the sum (SSS) and the carry-out.

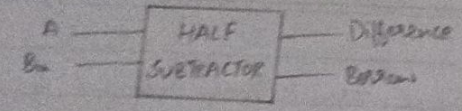
* Sum (S): The sum output is the result of the XOR operation between the three input bits.

$S = A \oplus B \oplus Cin$

* Carry-out (Cout): The carry-out is the result of the OR operation between the AND results of the input pairs.

$Cout = (A.B) + (Cin.(A \oplus B))$

Half subtractor



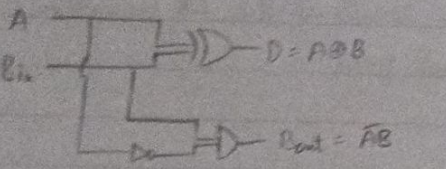
Truth Table:

A	Bin	D	Bout
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

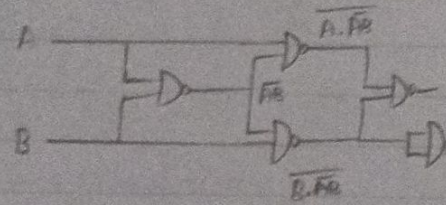
Difference $= A\bar{B} + B\bar{A}$

$= A \oplus B$

Borrow $= \bar{A}B$



Implementation of half subtractor using only NAND gates



$D = \overline{(A.\bar{A}B).(B.\bar{A}B)} = A \oplus B$

$B_{out} = \overline{\overline{B.\bar{A}B}} = \overline{\bar{B} + \bar{A}B}$

$= \bar{B} + AB$

$= \bar{B} . \overline{AB}$

$= \bar{A}B$

Expt. No. 2c Date / / Page No. 11

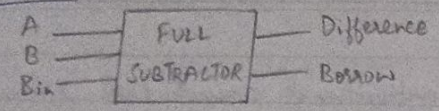
Name of Experiment..... Experiment Result .....

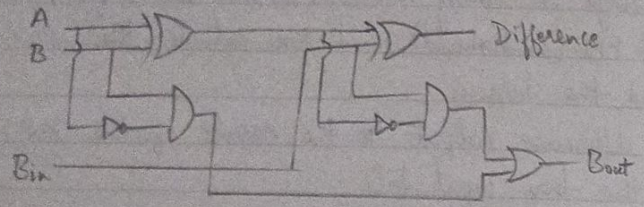
(2) Half Subtractor

A half subtractor is a combinational circuit that performs the subtraction of two single-bit binary numbers. It has two inputs, AAA (minuend) and BBB (subtrahend), and two outputs: the difference (DDD) and the borrow.

* Difference (D): The difference output is the result of the XOR operation between the two input bits.
  $D = A \oplus B$
* Borrow (Bout): The borrow output is the result of the AND operation between the inverted minuend and the subtrahend.
  $B_{out} = A \cdot B$

Full subtractor

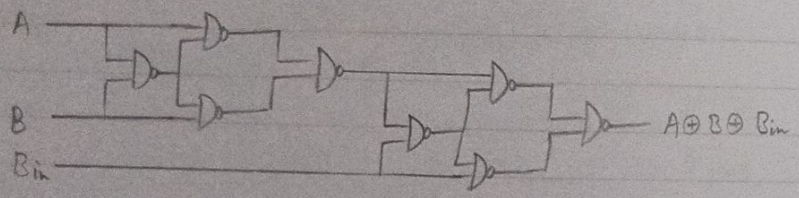




Truth table:

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Implementation of full subtractor using only NAND gates



Expt. No. 2d Date / / Page No. 12

Name of Experiment..... Experiment Result .....

(d) Full subtractor

A full subtractor is a combinational circuit that performs the subtraction of three single-bit binary numbers: the minuend (AAA), subtrahend (BBB), and borrow-in (Bin). It has three inputs and two outputs: the difference (DDD) and the borrow-out (Bout).

* Difference (D): The difference output is the result of the XOR operation between the three input bits.

$D = A \oplus B \oplus Bin$

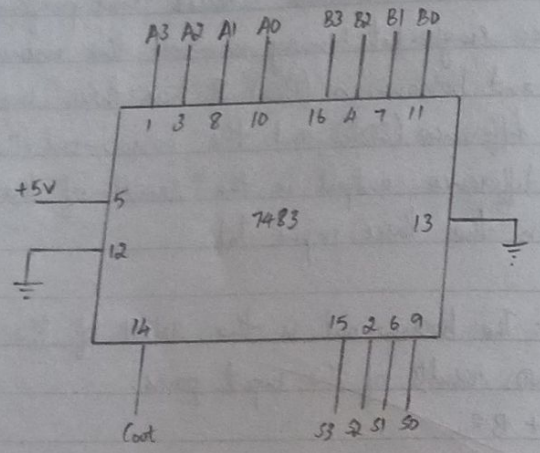
* Borrow-out (Bout): The borrow-out is the result of the OR operation between the AND results of the input pairs.

$Bout = \bar{A}(B \oplus Bin) + B\,Bin$

29/10/25

Circuit Diagram:

4-bit adder circuit using only IC7483



A3-A0	B3-B0	Sum (S3-S0)	Carry Out
1001	0101	1110	0
1111	1111	1110	1
1000	1001	0001	1
0101	0011	1000	0

Expt. No. 3a Date 5/11/25 Page No. 13

Name of Experiment..... Experiment Result .....

LAB CYCLE-3

Aim: To set up a 4-bit binary parallel adder/subtractor using IC7483.

Components required: IC 7483, Digital IC trainer kit, patch cords

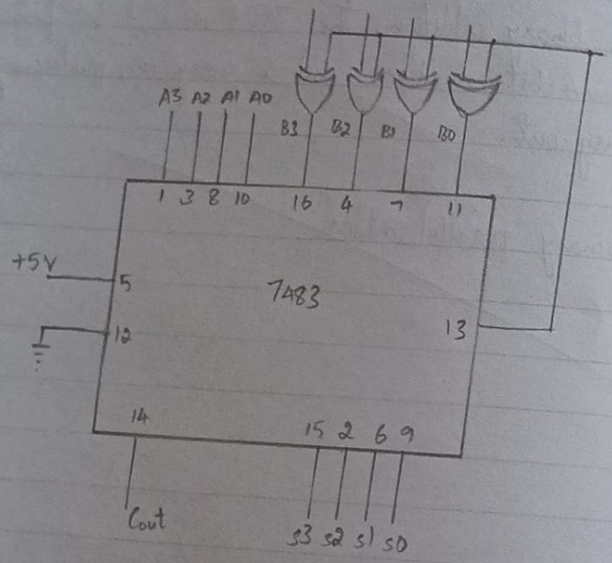
Theory: A parallel adder using the IC 7483 is a fast digital circuit designed for binary addition. The 7483 is a 4-bit full-adder IC that can add two 4-bit numbers and a carry-in, producing a 4-bit sum and a carry-out.

a) A 4-bit binary parallel adder.

Using only IC7483

A3 - A0	B3 - B0	Sum (S3 - S0)	Carry Out
1001	1011	0100	1
1111	0001	0000	1
0100	0110	1010	0
1100	0011	1111	0

Using IC7483 and IC7486



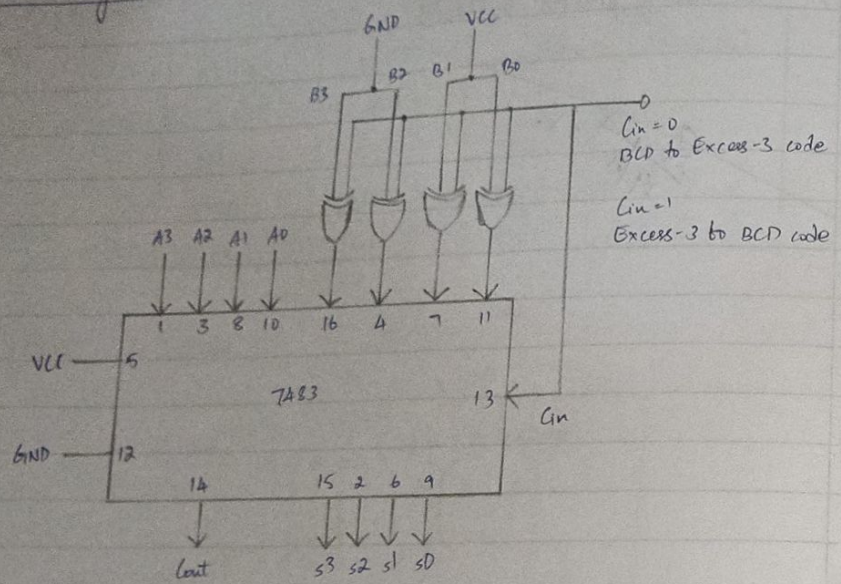
A3 - A0	B3 - B0	Carry In	Sum (S3 - S0)	Carry Out
1000	0101	1	0011	1
0100	0011	1	0001	1
1100	1011	1	1111	0
1111	1111	1	0000	1

Expt. No. 36 Date 5/11/05 Page No. 14

Name of Experiment..... Experiment Result .....

b) A 4-bit binary parallel subtractor

Circuit Diagram:



Expt. No. 4a Date 19/11/25 Page No. 15

Name of Experiment..... Experiment Result .....

LAB CYCLE - 4

Aim: To design and realize the following using IC 7483

(i) BCD to Excess-3 Code

(ii) Excess-3 to BCD Code

Components required: IC 7483, IC 7486, Patch cords and IC Trainer Kit.

Theory: Code converter is a combinational circuit that translates the input code word into a new corresponding word. The excess-3 code digit is obtained by adding three to the corresponding BCD digit. To construct a BCD-to-excess-3-code converter with a 4-bit adder feed BCD code to the 4-bit adder as the first operand and then feed constant 3 as the second operand. The output is the corresponding excess-3 code.

To make it work as a excess-3 to BCD converter, we feed excess-3 code as the first operand and then feed 2's complement of 3 as the second operand. The output is the BCD code.

Procedure:

* Check all the components for their working.
* Insert the appropriate IC into the IC base.
* Make connections as shown in the circuit diagram.
* Apply BCD code as first operand (A) and binary 3 as second operand (B) and cin = 0 for realizing BCD-to-Excess-3-code.

Truth table

(i) BCD to Excess-3 Code

BCD				Excess-3			
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

(ii) Excess-3 to BCD code

Excess-3				BCD			
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1

Expt. No. 4a Date 19/11/25 Page No. 16

Name of Experiment..... Experiment Result .....

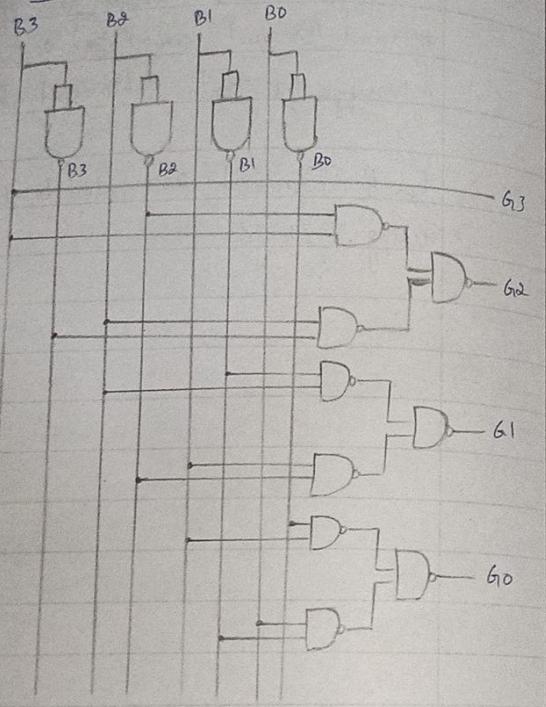
* Apply Excess-3-code as first operand (A) and binary 3 as second operand (B) and cin = 1 for realizing Excess-3-code to BCD.
* Verify the truth table and observe the outputs.

Result: Realized BCD code to Excess-3 code conversion and vice versa using 7483 IC.

(i) Binary to Gray Conversion

Binary				Gray			
B3	B2	B1	B0	G3	G2	G1	G0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Realization using NAND gates:



Boolean Expressions:

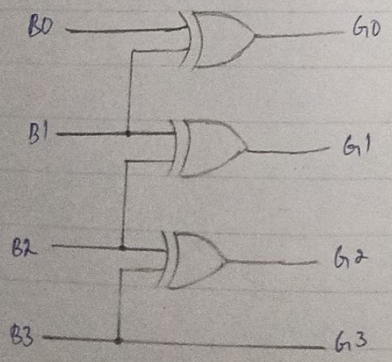
$G3 = B3$

$G2 = B3 \oplus B2$

$G1 = B1 \oplus B2$

$G0 = B1 \oplus B0$

Using EX-OR gates:



Expt. No. 4b Date 19/11/25 Page No. 17

Name of Experiment..... Experiment Result .....

Aim: To realize Binary to Gray code converter and vice versa.

Components required: IC 7400, IC 7486, IC 7408, Patch Cords and IC Trainer Kit.

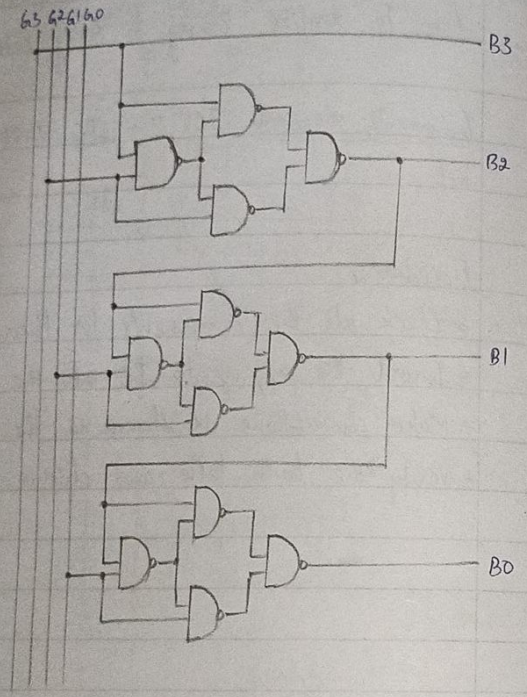
Procedure:

* Check all the components for their working
* Insert the appropriate IC into the IC base.
* Make connections as shown in the circuit diagram.
* Verify the truth table and observe the outputs.

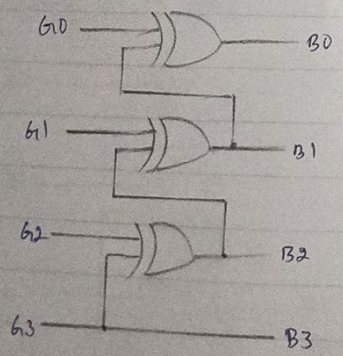
(ii) Gray to Binary Conversion

Gray				Binary			
G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

Realization using NAND gates:



Using EX-OR gates:



Boolean Expressions:

$B3 = G3$

$B2 = G3 \oplus G2$

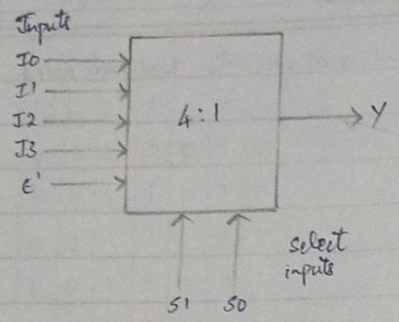
$B1 = G3 \oplus G2 \oplus G1$

$B0 = G3 \oplus G2 \oplus G1 \oplus G0$

Expt. No. 4b Date 19/11/25 Page No. 18

Name of Experiment..... Experiment Result .....

Result: Binary to gray code conversion and vice versa is realized using EX-OR gates and NAND gates.

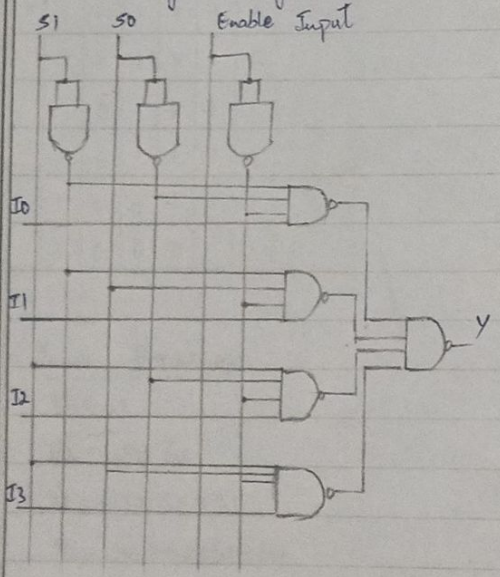


Output, $Y = E'S_1'S_0'I_0 + E'S_1'S_0I_1 + E'S_1S_0'I_2 + E'S_1S_0I_3$

Truth Table:

Select Inputs		Enable Input	Inputs				Outputs
S_1	S_0	E	I_0	I_1	I_2	I_3	Y
x	x	1	x	x	x	x	0
0	0	0	0	x	x	x	0
0	0	0	1	x	x	x	1
0	1	0	x	0	x	x	0
0	1	0	x	1	x	x	1
1	0	0	x	x	0	x	0
1	0	0	x	x	1	x	1
1	1	0	x	x	x	0	0
1	1	0	x	x	x	1	1

Realization using NAND gates



IC used:

7400 - 2 i/p NAND

7410 - 3 i/p NAND

7420 - 4 i/p NAND

Expt. No. 5a Date 26/11/25 Page No. 19

Name of Experiment..... Experiment Result .....

LAB CYCLE - 5

Aim: To design and implement 4:1 Multiplexer (MUX) using only NAND gates.

Components required: IC 7400, patch cords, trainer kit

Theory: A multiplexer is a circuit with many inputs but only one output. By using control signals (select lines) we can select any input to the output. Multiplexer is also called as data selector because the output bit depends on the input data that is selected. The general multiplexer circuit has 2^n input signals, n control/select signals and 1 output signal.

Procedure:

* Mount the required ICs on the trainer kit
* Make the connections according to the circuit diagram and design the truth table.
* Verify the result according to the truth table.

K-Map:

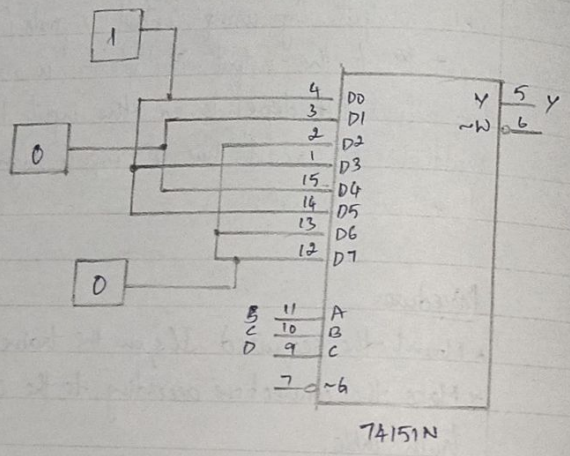
a → data line, b,c,d → select lines

	$\bar{b}\bar{c}\bar{d}$	$\bar{b}\bar{c}d$	$\bar{b}c\bar{d}$	$\bar{b}cd$	$b\bar{c}\bar{d}$	$b\bar{c}d$	$bc\bar{d}$	$\bar{b}\bar{c}\bar{d}$
a	0	1	2	3	4	5	6	7
$\bar{a}$	8	9	10	11	12	13	14	15
y	1	0	a	1	0	1	a	a
	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7

Truth table:

a	b	c	d	Y	$\bar{Y}$
0	0	0	0	1	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	1	0
1	0	1	1	1	0

Circuit diagram:



Expt. No. 5b Date 26/11/25 Page No. 20

Name of Experiment..... Experiment Result .....

Aim: To realize the given boolean expression using 8:1 multiplexer.

Expression: $f(a,b,c,d) = \Sigma m(0,3,5,8,10,11,13,14,15)$

Components required: IC 74151, patch cords, trainer kit

Theory: In the 8:1 multiplexer there are a total of eight inputs ($A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_7$). Three select lines ($S_0, S_1, S_2$) and single output (Y) on the basis of the combination of inputs, that are present at the select lines (S_0, S_1, S_2) and one of eight inputs are connected to the output.

Procedure:

* Analyze the given boolean expression.
* Get the simplified version of boolean expression and make circuits using the gates.
* Make the connections according to the circuits and verify the truth table.

Result: 8:1 MUX was introduced successfully through IC 74151 and verified successfully through the truth table.

Half Adder using MUX

Design:

SUM

I0	I1
0	1
2	3
A	A'

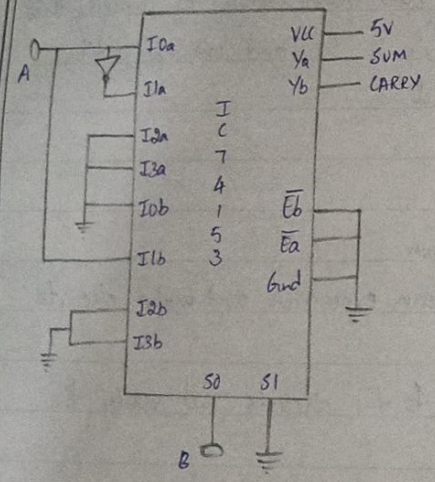
CARRY

I0	I1
0	1
2	3
0	A

Truth Table:

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Circuit:



Full Adder using MUX

Design:

SUM

I0	I1	I2	I3
0	1	2	3
4	5	6	7
A	A'	A'	A

CARRY

I0	I1	I2	I3
0	1	2	3
A	5	6	7
0	A	A	1

Expt. No. 6 Date 3/12/25 Page No. 21

Name of Experiment ..... Experiment Result .....

Aim: To set up a Half/Full Adder and Half/Full Subtractor using IC 74153.

Components required: IC 74153, patch cords, trainer kit

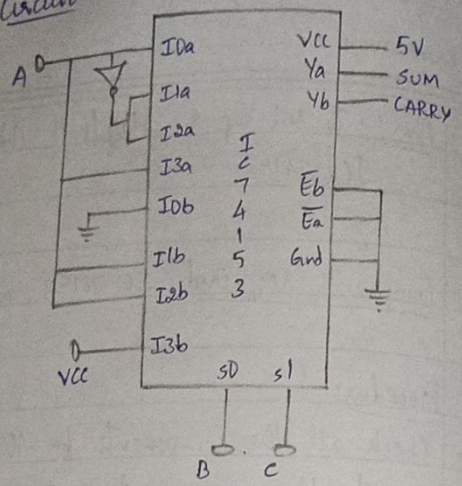
Procedure:

* Check all the components for their working.
* Insert the appropriate IC into the IC base.
* Make connections as shown in the circuit diagram.
* Verify the truth table and observe the outputs.

Truth Table:

Inputs			Outputs	
A	B	C	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit:



Half Subtracter using MUX:

Design:

DIFFERENCE

I0	I1
0	1
2	3
A	A'

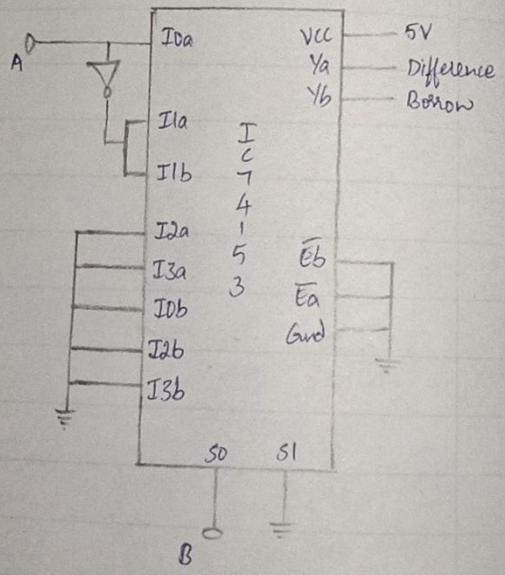
BORROW

I0	I1
0	1
2	3
0	A'

Truth Table:

Inputs		Outputs	
A	B	D	Br
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Circuit:



Expt. No. 6 Date 3/12/85 Page

Name of Experiment..... Experiment Result .....

Full Subtractor using MUX

Design:

DIFFERENCE

I0	I1	I2	I3
0	1	2	3
4	5	6	7
A	A'	A'	A

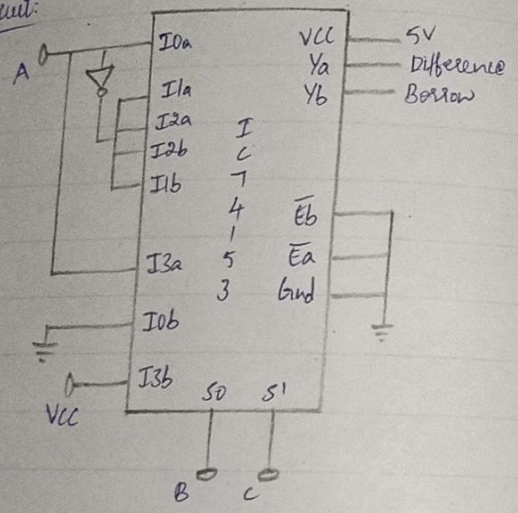
BORROW

I0	I1	I2	I3
0	1	2	3
4	5	6	7
0	A'	A	1

Truth Table:

Inputs			Outputs	
A	B	C	D	Br
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Circuit:



Expt. No. 6 Date 3/12/25 Page No. 23

Name of Experiment..... Experiment Result .....

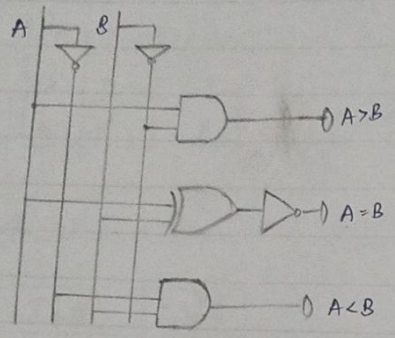
Result: Adder and subtractor circuits are realized using multiplexer IC 74153.

1) 1-Bit Comparator

$A>B = A\bar{B}$

$A<B = \bar{A}B$

$A=B = \bar{A}\bar{B} + AB$



Truth Table

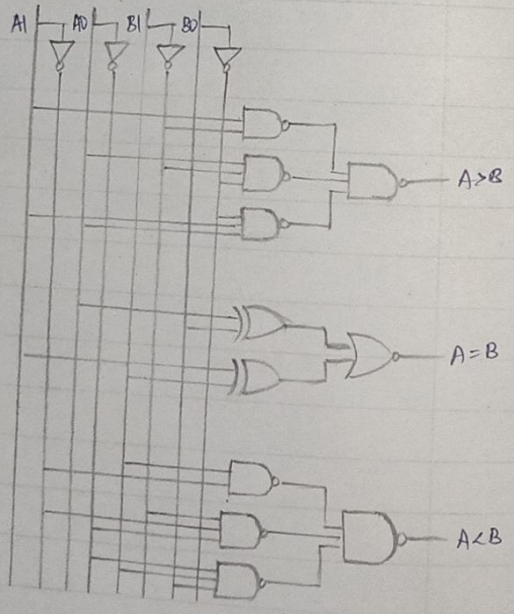
INPUTS		OUTPUTS		
A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

2) 2-Bit Comparator

$(A>B) = A_1\bar{B}_1 + A_0\bar{B}_1\bar{B}_0 + \bar{B}_0A_1A_0$

$(A=B) = (A_0 \oplus B_0)(A_1 \oplus B_1)$

$(A<B) = B_1\bar{A}_1 + B_0\bar{A}_1\bar{A}_0 + \bar{A}_0B_1B_0$



Expt. No. 7 Date 10 / 12 / 25 Page No. 24

Name of Experiment..... Experiment Result .....

LAB CYCLE -7

Aim: To realize One and Two Bit Comparator and study of 7485 magnitude comparator.

Theory: Magnitude Comparator is a logical circuit, which compares two signals A and B and generates three logical outputs, whether A>B, A=B, or A<B. IC 7485 is a high speed 4-bit Magnitude comparator, which compares two 4-bit words. The A=B Input must be held high for proper compare operation.

Components Required: IC 7400, IC 7410, IC 7432, IC 7486, IC 7402, IC 7408, IC 7404, IC 7485, Patch Cords and IC Trainer Kit.

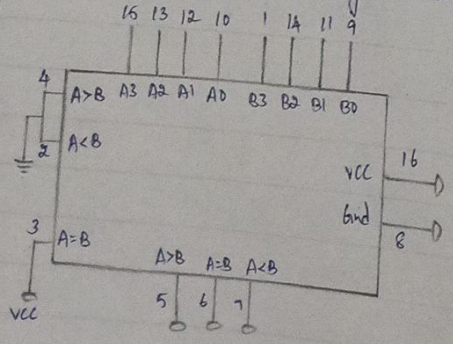
Procedure:

* Check all the components for their working.
* Insert the appropriate IC into the IC base.
* Make connections as shown in the circuit diagram
* Verify the Truth Table and observe the outputs.

Truth Table:

INPUTS				OUTPUTS		
A_1	A_0	B_1	B_0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

3) To compare the given data using 7485 chip



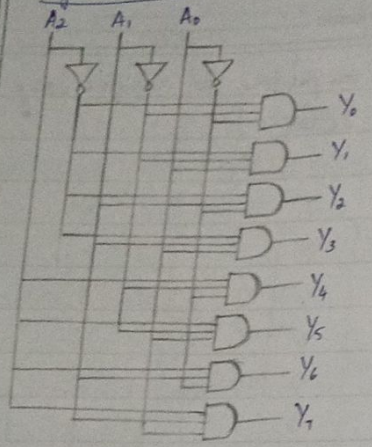
A				B				Result
A3	A2	A1	A0	B3	B2	B1	B0	
0	0	0	1	0	0	0	0	A>B
0	0	0	1	0	0	0	1	A=B
0	0	0	0	0	0	0	1	A<B

Expt. No. 7 Date 10 / 12 / 25 Page No. 25

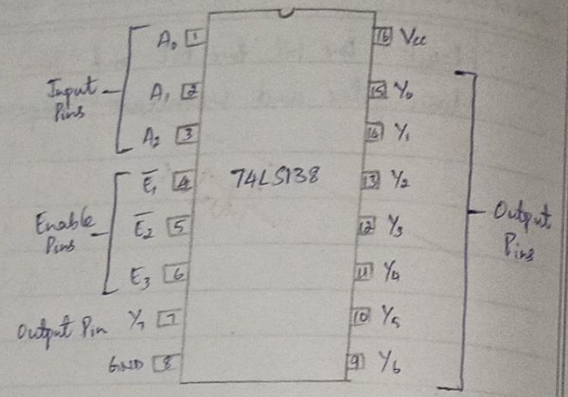
Name of Experiment..... Experiment Result .....

Result: One bit, two bit and four bit comparators are verified using basic gates and magnitude comparator IC7485.

Logical circuit



IC 74LS138 Pin Diagram



IC 74138 Truth Table

INPUTS						OUTPUTS							
$\overline{E_1}$	$\overline{E_2}$	E_3	A_0	A_1	A_2	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

H - High, L - Low, X - Don't Care

Expt. No. 8a Date 17/12/25 Page No. 26

Name of Experiment..... Experiment Result .....

LAB CYCLE-8

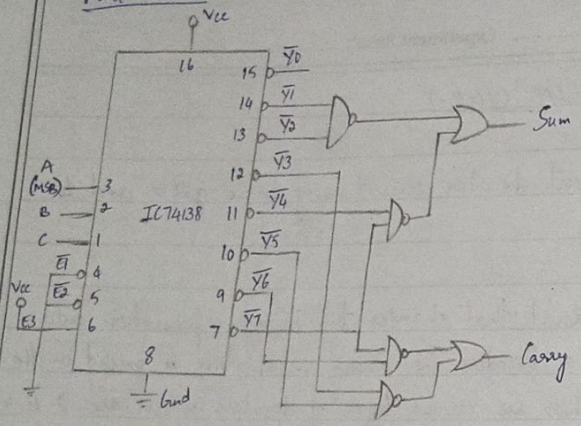
Aim: To design and implement decoder circuit using basic gates and to verify using IC 74LS138.

Theory: The combinational circuit that changes the binary information into 2^N output lines is known as Decoders. The binary information is passed in the form of N input lines. There are various types of decoders which are 2-to-4 line decoder, 3-to-8 decoder and 4-to-16 line decoder.

The 3-to-8 line decoder is also known as Binary to Octal Decoder. In a 3-to-8 line decoder, there is a total of eight outputs, i.e. Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7 and three outputs, i.e. A_0, A_1 and A_2.

Result: Verified the operation of 3-to-8 decoder

Full adder

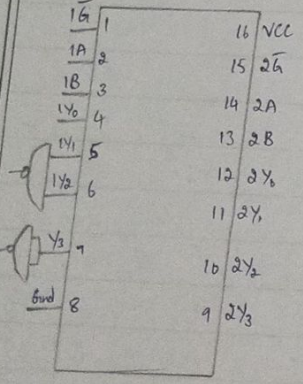


Inputs			Outputs	
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

For SUM = $\sum m(1,2,4,7)$

For CARRY = $\sum m(3,5,6,7)$

Half adder



A	B	Y_0	Y_1	Y_2	Y_3	Sum	Carry
0	0	1	0	0	0	0	0
0	1	0	1	0	0	1	0
1	0	0	0	1	0	1	0
1	1	0	0	0	1	0	1

$Y_0 = \bar{A}\bar{B}$ $Y_2 = A\bar{B}$

$Y_1 = \bar{A}B$ $Y_3 = AB$

$Sum = A \oplus B$

$Carry = AB$

Expt. No. 8b Date 17/12/25 Page No. 27

Name of Experiment..... Experiment Result .....

Aim: To design and implement half/full adder and half/full subtractor using IC74LS139.

Theory:

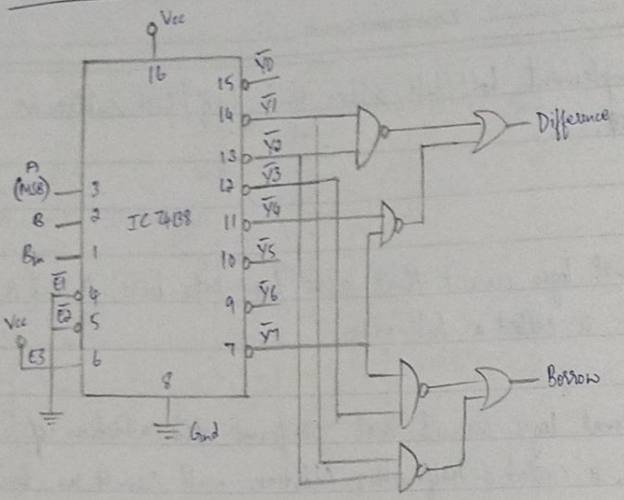
* Full Adder: A combinational logic circuit that adds two data bits, A and B, and a carry-in bit, Cin, is called a full-adder.

* Half Adder: A combinational logic circuit that performs the addition of two data bits, A and B, is called a half-adder. Addition will result in two output bits; sum bit and carry bit.

* Full Subtractor: Subtracting two single-bit binary values, B, Cin from a single-bit value. A produces a difference bit D and a borrow bit B_o. This is called full subtraction.

* Half Subtractor: Subtracting a single-bit binary value B from another A (i.e. A-B) produces a difference bit D and a borrow out bit Bout. This operation is called half subtraction.

Full subtractor

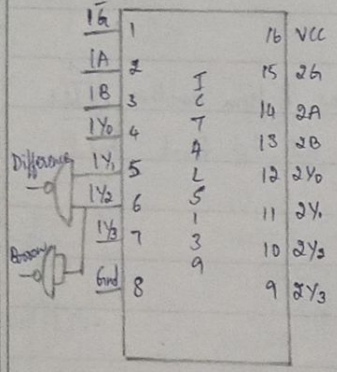


A	B	Bin	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

For Difference = $\Sigma m\,(1,2,4,7)$

For Borrow = $\Sigma m\,(1,2,3,7)$

Half subtractor



A	B	Y_0	Y_1	Y_2	Y_3	Difference	Borrow
0	0	1	0	0	0	0	0
0	1	0	1	0	0	1	1
1	0	0	0	1	0	1	0
1	1	0	0	0	1	0	0

$Y_0 = \bar{A}\bar{B}$ $\quad Y_2 = A\bar{B}$

$Y_1 = \bar{A}B$ $\quad Y_3 = AB$

Difference = $A \oplus B$

Borrow = $\bar{A}B$

Expt. No. 8b Date 17 / 12 / 25 Page No. 28

Name of Experiment..... Experiment Result .....

Result: Implemented full/half adder and full/half subtracter.

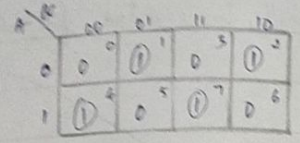
24/12/25

Even Parity Generator

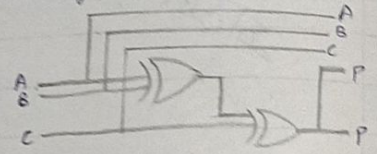
Truth Table

3-bit message			Even parity bit generated
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

K-map for 3-bit Even Parity Generator



Logic Circuit of Even Parity Generator

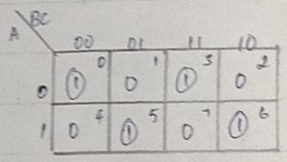


Odd Parity Generator

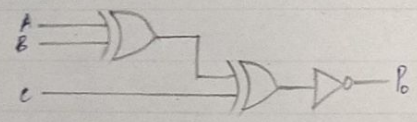
Truth Table

3-bit message			Odd parity bit generated(P)
A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

K-map for 3-bit Odd Parity Generator



Logic Circuit of Odd Parity Generator



Expt. No. 9a Date 24/12/25 Page No. 29

Name of Experiment ..... Experiment Result .....

LAB CYCLE-9

Aim: To design and implement 3-bit parity generator for odd and even parity and verify using Parity Checker.

Components required: IC 7486 (XOR gate), IC 7404 (NOT gate), digital IC Trainer Kit, Patch Chord.

Theory: A parity generator is a combinational logic circuit that generates the parity bit in the transmitter. On the other hand, a circuit that checks the parity in the receiver is called Parity Checker. A combined circuit or device of parity generators and parity checkers are commonly used in digital systems to detect the single bit errors in the transmitted data.

Even Parity Checker

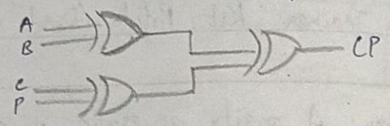
Truth Table of Even Parity Checker

4-bit received message				Parity error check C_p
A	B	C	P	
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

K-map for Even Parity Checker

AB \ CP	00	01	11	10
00	0 (0)	1 (1)	0 (3)	1 (2)
01	1 (4)	0 (5)	1 (7)	0 (6)
11	0 (12)	1 (13)	0 (15)	1 (14)
10	1 (8)	0 (9)	1 (11)	0 (10)

Logic Circuit of Even Parity Checker



Odd Parity Checker

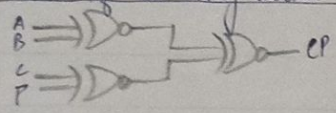
Truth Table of Odd Parity Checker

4-bit received message				Parity error check C_p
A	B	C	P	
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

K-map for Odd Parity Checker

AB \ CP	00	01	11	10
00	1 (0)	0 (1)	1 (3)	0 (2)
01	0 (4)	1 (5)	0 (7)	1 (6)
11	1 (12)	0 (13)	1 (15)	0 (14)
10	0 (8)	1 (9)	0 (11)	1 (10)

Logic Circuit of Odd Parity Checker

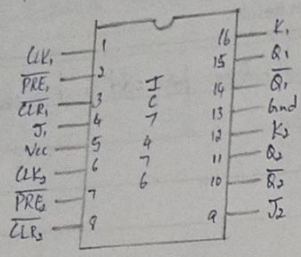


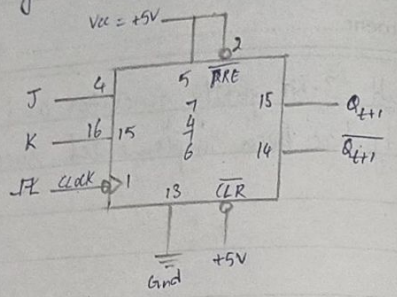
Expt. No. 9a Date 24 / 12 / 25 Page No. 30

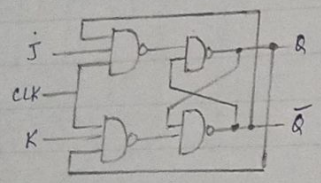
Name of Experiment..... Experiment Result .....

Result: 3-bit parity generator and parity checker for odd and even parity has been implemented successfully.

(i) Implementation of J-K Flip-Flop using NAND gates:

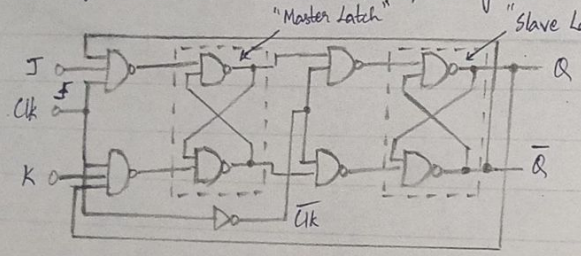






Inputs			Outputs	
CLK	J	K	Q	Q̄
↓	0	0	Q	Q̄
↓	1	0	1	0
↓	0	1	0	1
↓	1	1	Toggle	

(ii) Implementation of J-K Master Slave Flip-Flop using NAND gates:



Row No.	J	K	C	Q^+	$\bar{Q}^+$
0	0	0	↓	Q	Q̄
1	0	1	↓	0	1
2	1	0	↓	1	0
3	1	1	↓	Q̄	Q
4	X	X	0	Q	Q̄

Expt. No.. 9b Date 26/12/25 Page No. 31

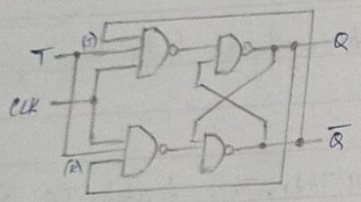
Name of Experiment..... Experiment Result .....

Aim: To implement the circuit and to verify the truth table of the following Flip-Flops

Components required: IC 7485, 7408, 8411, 7421, 7432, digital IC Trainer kit, patch chords.

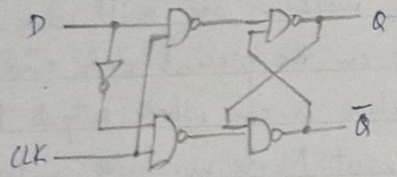
Theory: Basically Flip-Flops are the bistable multivibrators that stores logic 1 and logic 0. Shift registers, memory, and counters are built by using Flip-Flops. Any complex sequential machines are build using Flip-Flops. Sequential circuit output depends on the present state and input applied at that instant. Mealy Machine is one whose output depends on both the present state and the input. Moore machines is the one whose output depends only on the present state of the sequential circuit. Note that the truth table of J-K Flip-Flop is same as the Master-Slave J-K Flip-Flop.

(iii) Implementation of T Flip Flop Circuit using NAND gates:



T	Q	$\bar{Q}$
0	0	0
0	1	0
1	0	1
1	1	0

(iv) Implementation of D Flip Flop using NAND gates:



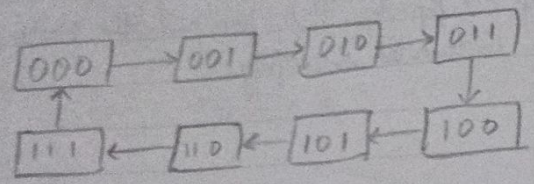
Q	D	$Q_{(t+1)}$
0	0	0
0	1	1
1	0	0
1	1	1

Expt. No. 9b Date 24 / 12 / 25 Page No. 32

Name of Experiment..... Experiment Result .....

Result: The JK flip-flop, T flip-flop, D-flip flop and the Master-Slave JK flip-flop has been implemented successfully.

Transition diagram:



K-map:

1	X	X	1
1	X	X	1

$J_0 = 1$

X	1	1	X
X	1	1	X

$K_0 = 1$

0	1	X	X
X	1	X	X

$J_1 = Q_0$

X	X	1	0
X	X	1	0

$K_1 = Q_0$

0	0	1	0
X	X	X	X

$J_2 = Q_1 Q_0$

X	X	X	X
0	0	1	0

$K_2 = Q_1 Q_0$

Expt. No. 10 Date 31/12/25 Page No. 33

Name of Experiment..... Experiment Result .....

LAB CYCLE-10

Aim: To design and test 3-bit synchronous counter using a flipflop IC7476 for the given sequence.

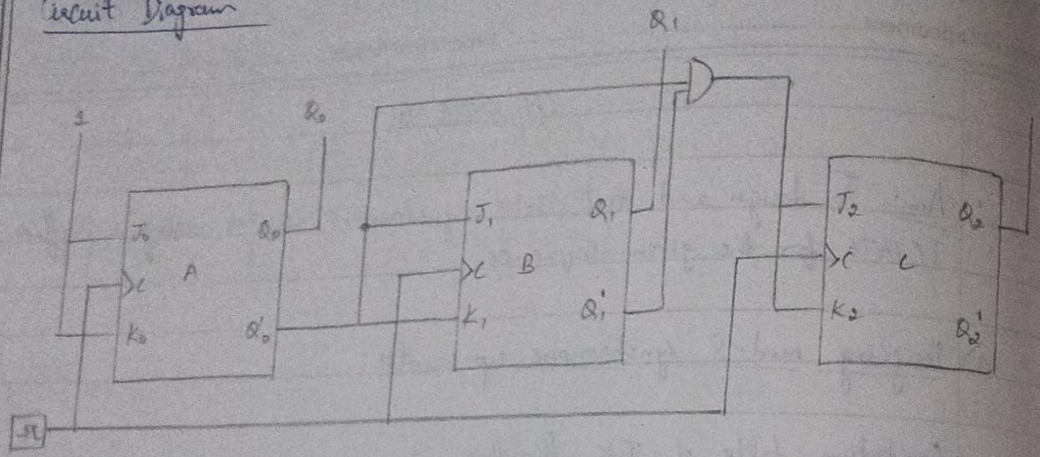
a) Designing mod-8 synchronous up counter:

Excitation table of J-K flip flop

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Present			Next			FFC		FFB		FFA	
Q_2	Q_1	Q_0	Q_2'	Q_1'	Q_0'	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

Circuit Diagram

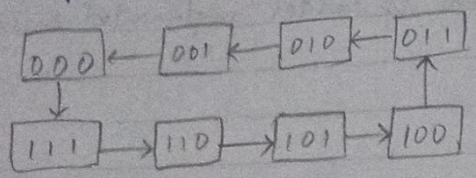


Expt. No. 10 Date 31 / 12 / 25 Page No. 34

Name of Experiment..... Experiment Result .....

Result: Mod-8 Synchronous Up counter implemented successfully.

Transition Diagram:



K-maps:

1	0	0	0
x	x	x	x

$J_2 = Q_1' Q_0'$

x	x	x	x
1	0	0	0

$K_2 = Q_1' Q_0'$

1	0	x	x
1	0	x	x

$J_1 = Q_0'$

x	x	0	1
x	x	0	1

$K_1 = Q_0'$

1	x	x	1
1	x	x	1

$J_0 = 1$

x	1	1	x
x	1	1	x

$K_0 = 1$

Expt. No. 10 Date 31/12/25 Page No. 35

Name of Experiment..... Experiment Result .....

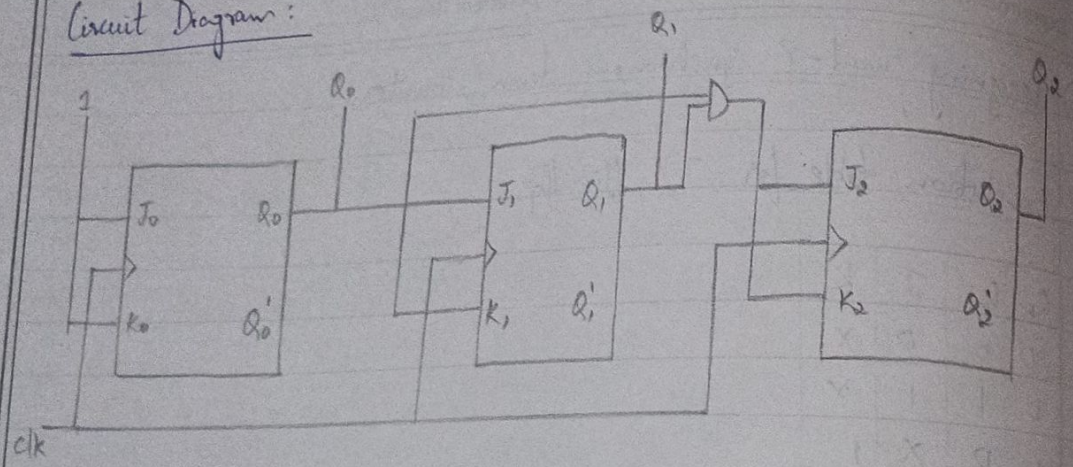
3) Designing mod-8 synchronous down counter.

Excitation table for J-K flip flop

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Present			Next			FFC		FFB		FFA	
Q_2	Q_1	Q_0	Q_2'	Q_1'	Q_0'	J_2	K_2	J_1	K_1	J_0	K_0
1	1	1	1	1	0	x	0	x	0	x	1
1	1	0	1	0	1	x	0	x	1	1	x
1	0	1	1	0	0	x	0	0	x	x	1
1	0	0	0	1	1	x	1	1	x	1	x
0	1	1	0	1	0	0	x	x	0	x	1
0	1	0	0	0	1	0	x	x	1	1	x
0	0	1	0	0	0	0	x	0	x	x	1
0	0	0	1	1	1	1	x	1	x	1	x

Circuit Diagram:



Expt. No. 10 Date 31/12/25 Page No. 36

Name of Experiment..... Experiment Result .....

Result: Synchronous Mod-8 down counter implemented succesfully